Advanced Digital Design

PRESENTED BY:
Dr. Shasanka Sekhar Rout

SYLLABUS

Session 1
Lecture: Basic Digital Circuits
- Basic Digital Circuits: Introduction, Principles, Working.
- Usage / Applications.

Session 2
Lecture: Combinational Circuits
- Logic Arrays
- NAND-NAND / NOR-NOR circuits
- Encoders and Decoders

Session 3
Lecture: Arithmetic Circuits
- 2's Complement
- Half and Full adders
- RCA,CLA,CSA

Session 4 & 5
Lecture: Logic Minimization
- Introduction to Various Logic Minimization techniques
- Quine McCluskey technique
- Shannon's reduction using relay switches

Session 6
Lecture: Logical Hazards
- Hazards and glitches
- Hazard elimination

Session 7
Lecture: Logic Families
- Introduction to different Logic families with their sub-families/types.
- TTL – NAND using TTL with working, different types of TTL families with features, usage.
- ECL – NAND using ECL with working, different types of ECL families with features, usage.
- CMOS – NAND using CMOS with working, Different types of CMOS families with features, usage.
- Comparison of speed and power of different families and sub-families/types.

Session 8
Lecture: Tristate and other circuits
- Tristate
- Muller-C
- AOI (And-Or-Inverter)
- Design of Logic Gates with CMOS technology: NOT, NAND, NOR, XOR.

Session 9
Lecture: Sequential Circuits
- ❖ Sequential Circuit Principles
- ❖ Types of Sequential Circuits
- ❖ Clock jitter, clock skew, timing parameters, propagation delays
- ❖ Latches
- ❖ Flip-flops

Session 10 & 11
Lecture: Counter Design and Frequency Division
- ❖ Counters: Introduction and types
- ❖ Designing of Counters using FSM approach: MOD-N, Binary, Decade
- ❖ Binary Counter as a Frequency divider
- ❖ Frequency division hazards

Session 12
Lecture: Registers
- ❖ Registers
- ❖ Shift Registers
- ❖ LIFO
- ❖ FIFO
- ❖ LFSR

Session 13 & 14
Lecture: Finite State Machines (FSM)
- ❖ Types of FSM
- ❖ Problem solving
- ❖ State reduction techniques

Session 15 & 16
Lecture: Static Timing Analysis
- ❖ Need of Static Timing Analysis of Digital Circuits
- ❖ Metastability
- ❖ Setup and Hold Violations
- ❖ Time Borrowing and Time Stealing

Session 17 & 18
Lecture: Asynchronous Circuits
- ❖ Introduction to multiple clock domain circuits
- ❖ Working across multiple clock domains
- ❖ Handshaking

Session 19 & 20
Lecture: Asynchronous to Synchronous Circuit Interaction

Session 21 & 24
Lecture: Case studies of Advanced Digital Design Circuits
- ❖ Several case studies like complex FSMs, Real life problems and their solutions, Pipelined Approaches for Communication Protocols or Recursive Operations, Design Optimization, Latency reduction, Development of Microporcessor / Microcontroller Design, Circuit Development for Data Encryption / Decryption Algorithms etc.

Session 9

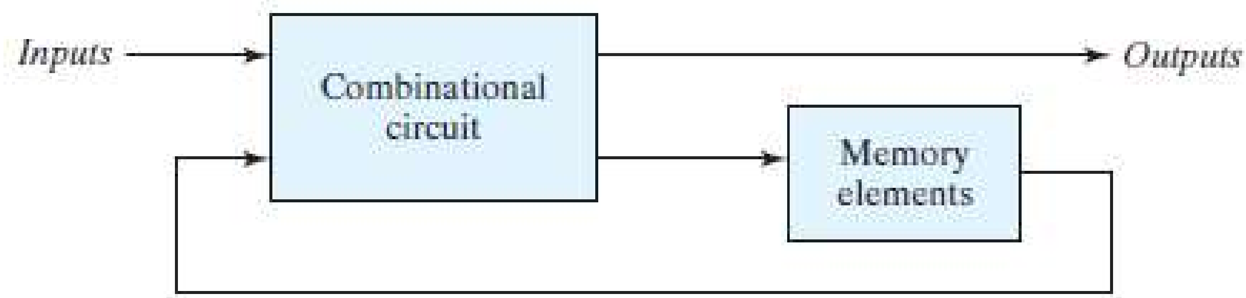
Lecture: Sequential Circuits

- ✓Sequential Circuit Principles
- ✓Types of Sequential Circuits
- ✓Clock jitter, clock skew, timing parameters, propagation delays
- ✓Latches
- ✓Flip-flops

Sequential Circuits

Sequential Circuit Principles:

- ✓ It consists of a combinational circuit to which storage elements are connected to form a feedback path.
- ✓ The block diagram demonstrates that the outputs in a sequential circuit are a function not only of the inputs, but also of the present state of the storage elements.



- ➢ Basic Components of Sequential Circuits are:
  - ✓ **Combinational Logic** – Determines the next state and output based on inputs and current state.
  - ✓ **Memory Elements (Flip-Flops or Latches)** – Store the circuit's current state.
  - ✓ **Clock Signal** – Synchronizes state changes.
  - ✓ **Feedback Paths** – Allow the circuit to use previous outputs as inputs.

Difference between Combinational and Sequential Circuits

Combinational Circuit

- A circuit whose output depends only upon the present inputs.
- These circuits do not have any memory element.
- There is no feedback between input and output.
- This is time independent.
- It does not have clock signal.
- Elementary building blocks: Logic gates.
- Used for arithmetic as well as Boolean operations.
- Speed is fast.
- It is designed easy.
- It is easy to use and handle.
- Examples: Encoder, Décoder, Multiplexer, De-multiplexer.

Sequential Circuit

- A circuit output depends upon the present inputs as well as past history of the inputs.
- These circuits have memory element.
- There exists a feedback path between input and output.
- This is time dependent.
- It may or may not clock signal but most sequential circuit have a clock signal.
- Elementary building blocks: Flip-flops.
- Mainly used for storing data.
- Speed is slow.
- It is designed tough as compared to combinational circuits.
- It is not easy to use and handle.
- Examples: Flip-flops, Counters, Shift registers.

Sequential Circuits

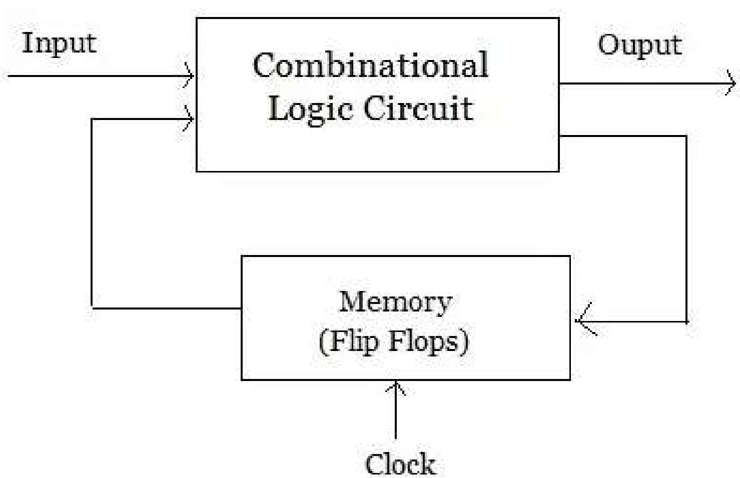
Classification of Sequential Circuits:

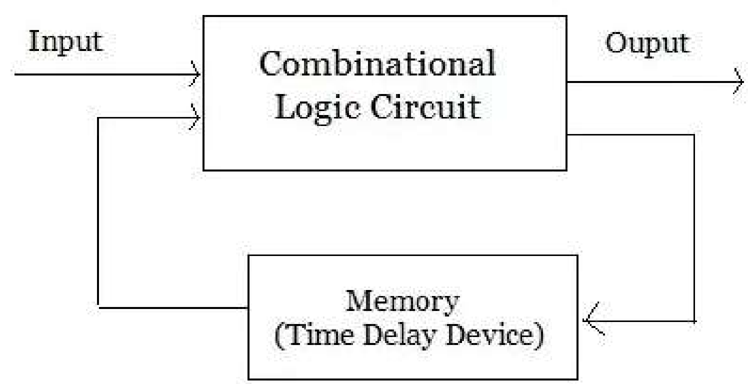
✅ **Synchronous Sequential Circuits**

- **Clocked flip-flops** update the state only at specific clock pulses.
- Predictable and stable behavior.
- Example: **Counters, Shift Registers, Finite State Machines (FSMs)**.

✅ **Asynchronous Sequential Circuits**

- State changes occur **immediately** based on input changes.
- Faster but more prone to race conditions.
- Example: **Muller C-element, Handshake Circuits**.





Sequential Circuits

Types of Sequential Circuits:

(a) Latches – Basic memory elements that change state asynchronously based on input changes.

- SR Latch
- D Latch

(b) Flip-Flops – Edge-triggered elements that change state only at clock edges.

- SR Flip-Flop
- D Flip-Flop
- JK Flip-Flop
- T Flip-Flop

(c) Registers – A collection of flip-flops used to store multiple bits.

- Shift Registers, Universal Registers

(d) Counters – Special registers used for counting operations.

- Binary Counters, Johnson Counters, Ring Counters

(e) Finite State Machines (FSMs) – Complex sequential circuits that transition between states.

- Mealy FSM (Output depends on Input + State)
- Moore FSM (Output depends only on State)

Sequential Circuits

Applications of Sequential Circuits:

✅ **Computers & Microprocessors** – Registers, memory, and control units.
✅ **Communication Systems** – Data buffering and error detection.
✅ **Digital Clocks & Timers** – Counters and dividers.
✅ **Automated Control Systems** – Traffic lights, vending machines, elevators.

✓ Sequential circuits are fundamental to VLSI design, enabling memory, control logic, and real-time processing.

Sequential Circuits

Clock jitter, clock skew, timing parameters, propagation delays:

Clock jitter: It refers to the **variability in the timing of clock edges** from their expected positions. It causes timing uncertainties in **sequential circuits**, affecting their reliability and performance.

Types of Clock Jitter:

✅ **Cycle-to-Cycle Jitter**

- Variation in the **period of consecutive clock cycles**.
- Affects real-time systems and high-speed processors.

✅ **Period Jitter**

- Deviation of the **clock period** from its expected value.
- Impacts timing closure in digital designs.

✅ **Phase Jitter**

- Short-term variations in clock phase, affecting **synchronous data transmission**.
- Important in **communication systems**.

✅ **Long-Term Jitter (Wander)**

- Slow variations in clock edges over a long duration.
- Affects **frequency stability** in high-precision applications.

Sequential Circuits

Causes of Clock Jitter:

🔹 **Power Supply Noise** – Variations in **VDD/GND** impact oscillator stability.
🔹 **Temperature Variations** – Affect transistor switching speed and clock skew.
🔹 **Interconnect Delays** – Variability in wire capacitance and resistance.
🔹 **Electromagnetic Interference (EMI)** – Crosstalk and noise from nearby circuits.
🔹 **Process Variations** – Manufacturing inconsistencies in VLSI fabrication.

Effects of Clock Jitter:

⚠ **Setup & Hold Violations** – Causes metastability in flip-flops.
⚠ **Data Corruption** – Leads to incorrect sampling in data communication.
⚠ **Reduced System Performance** – Affects high-speed processors and DSP systems.
⚠ **Increased Power Consumption** – Causes excessive transitions and glitches.

Clock Jitter Reduction Techniques:

✅ **Low-Jitter Clock Sources** – Use stable crystal oscillators (e.g., TCXO, OCXO).
✅ **Power Supply Filtering** – Minimize noise with decoupling capacitors and regulators.
✅ **Clock Tree Optimization** – Proper routing and buffering in clock distribution networks (CDN).
✅ **Phase-Locked Loops (PLLs)** – Jitter attenuation using feedback-controlled oscillators.
✅ **Shielding & Grounding** – Reducing electromagnetic interference.

Sequential Circuits

Clock skew: Clock skew refers to the **difference in arrival times of the clock signal** at different points in a synchronous circuit. It occurs due to **unequal path delays** in the clock distribution network and can cause timing violations in sequential circuits.

Types of Clock Skew:

✅ **Positive Skew**

- The clock **arrives earlier** at the destination register than at the source register.
- Can cause **setup time violations**, leading to incorrect data latching.

✅ **Negative Skew**

- The clock **arrives later** at the destination register than at the source register.
- Can cause **hold time violations**, leading to data instability.

✅ **Local Skew**

- Skew between **two flip-flops** in the same clock domain.
- Affects local data propagation.

✅ **Global Skew**

- Skew between **different clock domains** or across large chip areas.
- Affects overall system synchronization.

Sequential Circuits

Causes of Clock Skew:

- 🔷 **Unequal Wire Delays** – Variations in clock trace lengths in PCBs or IC layouts.
- 🔷 **Process Variations** – Fabrication inconsistencies affecting transistor performance.
- 🔷 **Temperature Variations** – Changes in wire resistance and capacitance.
- 🔷 **Load Imbalance** – Uneven fan-out of clock buffers.
- 🔷 **Jitter in Clock Distribution** – Random variations in clock edge timing.

Effects of Clock Skew:

⚠ **Setup Violations** – If the clock arrives **too early**, data may not be ready.
⚠ **Hold Violations** – If the clock arrives **too late**, old data may still be latched.
⚠ **Increased Power Consumption** – Extra buffers and repeaters add dynamic power loss.
⚠ **Functional Failures** – Incorrect timing can cause **metastability** in flip-flops.

Techniques to Minimize Clock Skew:

✅ **Balanced Clock Tree (H-Tree Network)** – Ensures equal path delays.
✅ **Clock Buffering** – Controlled buffer insertion to distribute the clock evenly.
✅ **Clock Gating Optimization** – Reducing unnecessary clock toggling to save power.
✅ **Delay Matching** – Adjusting wire lengths and buffer delays for synchronization.
✅ **Use of PLLs/DLLs** – Phase-locked loops (PLLs) and delay-locked loops (DLLs) dynamically adjust skew.
✅ **Temperature Compensation** – Adaptive tuning for environmental variations.

Sequential Circuits

Timing Parameters:

- Timing parameters define the constraints that ensure correct data propagation and synchronization in **sequential circuits**.
- They are crucial for **setup, hold, propagation delay, and overall performance** in **VLSI, FPGA, and ASIC design**.

1. Propagation Delay (tpd):

- The time taken for an input change to reflect at the output.
- Measured from **50% of input transition** to **50% of output transition**.

Types:

  - **tpLH** – Low-to-High propagation delay.
  - **tpHL** – High-to-Low propagation delay.

2. Setup Time (tsu):

- The **minimum time before the clock edge** that the input data must be stable.
- If violated, it causes **setup time violation**, leading to **incorrect data capture**.

3. Hold Time (th):

- The **minimum time after the clock edge** that the input data must remain stable.
- If violated, it causes **hold time violation**, leading to **metastability**.

Sequential Circuits

4. Clock-to-Q Delay (tCQ):

- The time taken for the **flip-flop's output (Q) to change** after a clock edge.
- Important for determining the maximum operating frequency of a sequential circuit.

5. Clock Period (T):

- The total time of **one full clock cycle** (from one rising edge to the next).

$$f = 1/T$$

6. Clock Skew (t_skew):

- The **difference in arrival times** of the clock signal at different registers.
- If skew is large, it can cause **timing violations**.

7. Clock Jitter:

- **Uncertainty in clock edge timing** due to variations in clock generation and distribution.
- Affects **setup and hold margins**, causing unreliable operation in high-speed circuits.

8 Maximum Clock Frequency (f_max):

- The highest frequency at which a circuit can operate **without setup violations**.

Sequential Circuits

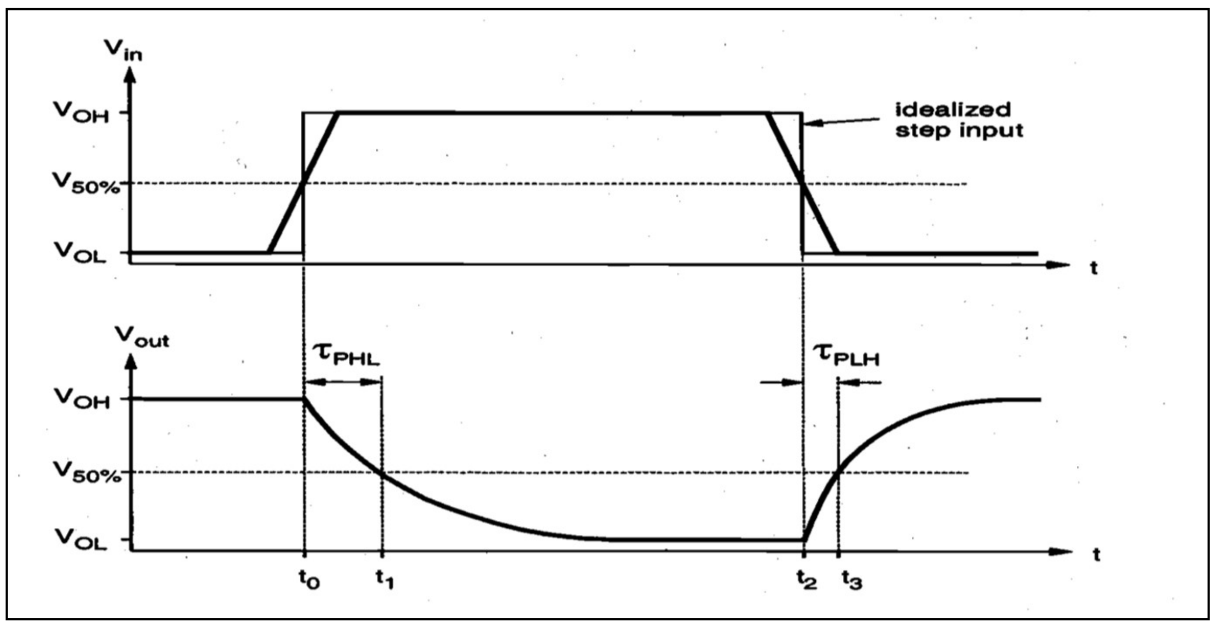
Propagation Delays:

- Propagation delay (**tpd**) is the **time taken** for a change in the input of a logic gate to produce a change in the output. It determines the **speed of digital circuits** and affects **timing constraints in VLSI design**.
- **Propagation Delay (tpd):** The time interval between the **50% transition point** of the input signal and the **50% transition point** of the output signal.

✓ tpLH (Low-to-High delay): Delay when output transitions from LOW (0) to HIGH (1).

✓ tpHL (High-to-Low delay): Delay when output transitions from HIGH (1) to LOW (0).

$$\tau_P = \frac{\tau_{PHL} + \tau_{PLH}}{2}$$



Sequential Circuits

Types of Propagation Delays:

✅ **Gate Delay:** The delay introduced by a **logic gate** due to transistor switching time.
✅ **Interconnect Delay:** Delay caused by **resistance and capacitance (RC) of interconnecting wires**.
✅ **Clock-to-Q Delay (tCQ):** Time taken for a **flip-flop's output (Q) to change** after a clock edge.
✅ **Data Path Delay:** The cumulative delay through **a sequence of logic gates** in a circuit.

Factors Affecting Propagation Delay:

- **Process Variation** – Manufacturing differences affect transistor characteristics.
- **Load Capacitance (CL)** – Higher capacitance increases the switching time.
- **Supply Voltage (VDD)** – Lower voltage slows down transistor switching.
- **Temperature Changes** – Higher temperatures increase resistance, slowing down circuits.
- **Fan-Out** – More connected gates increase delay due to additional load.

How to Minimize Propagation Delay:

✅ **Use Faster Transistors** – Reduce gate delay by using **low-power, high-speed CMOS**.
✅ **Reduce Load Capacitance** – Optimize interconnect and buffer placement.
✅ **Optimize Power Supply (VDD)** – Higher voltage speeds up transitions.
✅ **Improve Clock Distribution** – Use **H-tree or PLL-based clock networks**.
✅ **Use Pipelining and Parallel Processing** – Break long delay paths into smaller sections.

Sequential Circuits

Latches:

- ✓ A **latch** is a **sequential logic circuit** that stores **one bit of data** and operates based on the input and enable/control signals.

- ✓ latches are **level-triggered**, meaning they change their output **whenever the enable signal is active**, rather than on clock edges

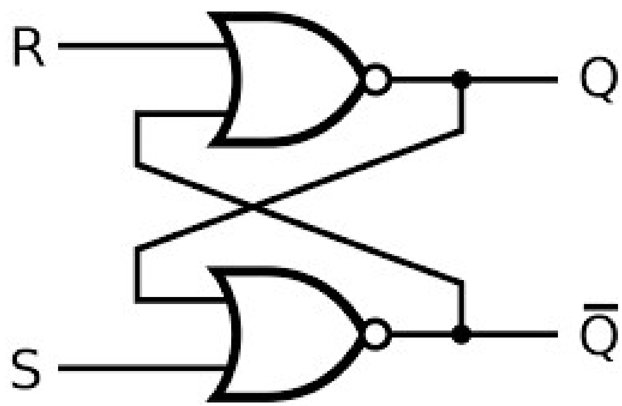
- ✓ Latch is an electronic logic circuit with two stable states namely high output as well as low output i.e. it is a **bistable multivibrator**.

- ✓ The latch has two useful states.
  - ✓ When output $Q = 1$ and $Q' = 0$, the latch is said to be in the ***set state***.
  - ✓ When $Q = 0$ and $Q' = 1$, it is in the ***reset state***.

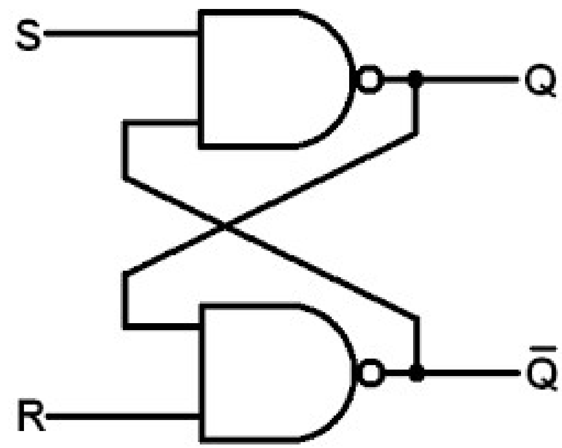
Sequential Circuits

SR Latch:

✓ The *SR* latch is a circuit with two cross-coupled NOR gates or two cross-coupled NAND gates, and two inputs labeled *S* for set and *R* for reset.



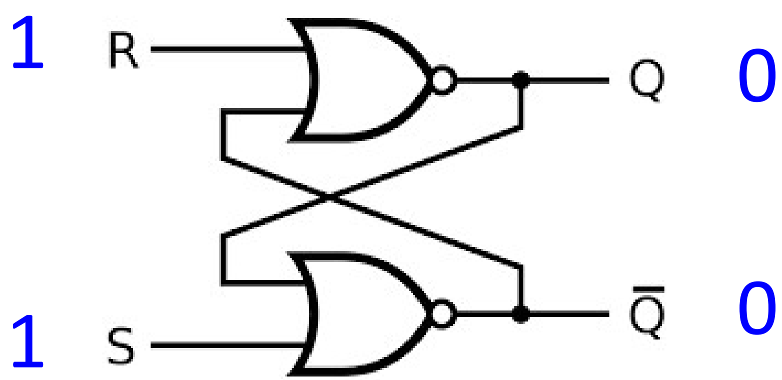
***SR* latch with NOR gates**



***SR* latch with NAND gates**

Sequential Circuits

SR latch with NOR gates:



Logic diagram

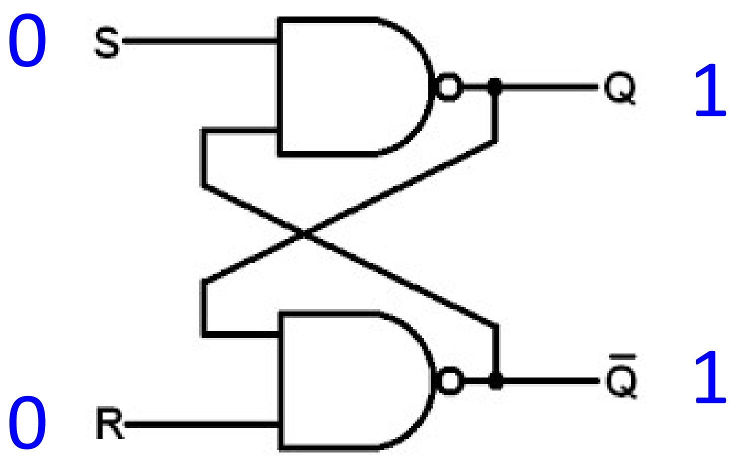
S	R	Q	$\overline{Q}$	
0	0	0	1	*No change*
		1	0	*No change*
0	1	0	1	*Reset*
1	0	1	0	*Set*
1	1	0	0	*Indeterminate*

Function table

Input		Output
A	B	$\overline{A+B}$
0	0	1
0	1	0
1	0	0
1	1	0

Sequential Circuits

SR latch with NAND gates:



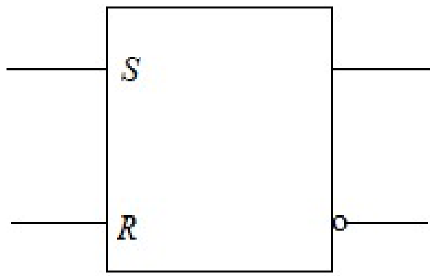
Logic diagram

S	R	Q	$\overline{Q}$	
0	0	1	1	*Indeterminate*
0	1	1	0	*Set*
1	0	0	1	*Reset*
1	1	1	0	*No change*
		0	1	*No change*

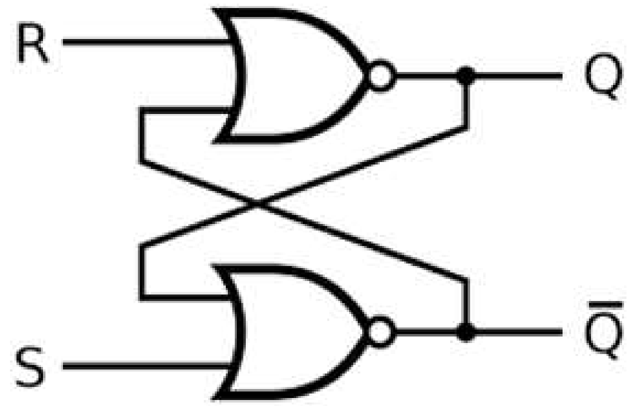
Function table

Input		Output
A	B	Y= $\overline{A.B}$
0	0	1
0	1	1
1	0	1
1	1	0

Sequential Circuits



(a) Graphic symbol

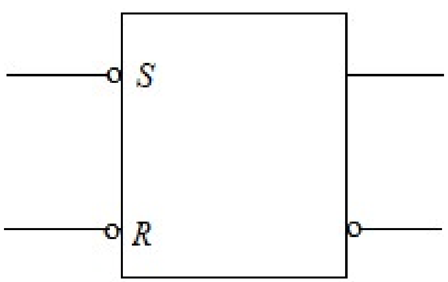


(b) Logic diagram

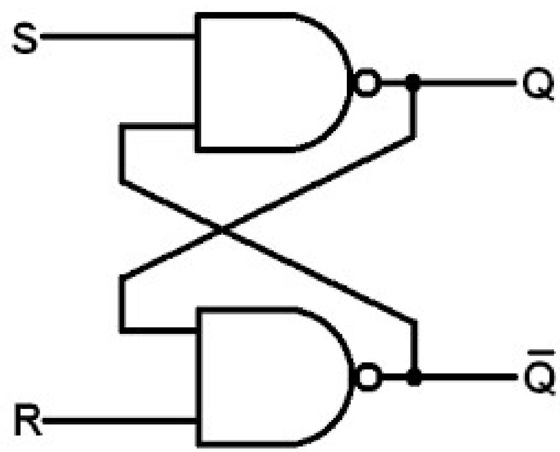
S	R	Q	$\overline{Q}$	
0	0	Q	Q'	*No change*
0	1	0	1	*Reset*
1	0	1	0	*Set*
1	1	0	0	*Indeterminate*

(c) Function table

SR latch with NOR gates



(a) Graphic symbol



(b) Logic diagram

S	R	Q	$\overline{Q}$	
0	0	1	1	*Indeterminate*
0	1	1	0	*Set*
1	0	0	1	*Reset*
1	1	Q	Q'	*No change*

(c) Function table

SR latch with NAND gates

Sequential Circuits

D (Data/Delay) Latch:

✅ Overcomes invalid state in SR latch
✅ Single input (D), which directly controls output
✅ When Enable = 1, Output follows D

❖ Use Case: Data storage, register circuits, and memory design.

Truth Table

Enable (E)	D	Q (Output)
0	X	Q (Previous)
1	0	0
1	1	1

JK Latch:

✅ Solves the invalid state issue of SR latch
✅ Behaves like an SR latch, but toggles output when both J & K are HIGH

🔷 Used in: Frequency dividers, counters, and sequential circuits.

Truth Table

J	K	Q (Next State)
0	0	Q (Previous)
0	1	0
1	0	1
1	1	**Toggles (Q → Q')**

Sequential Circuits

T (Toggle) Latch:

- ✅ Single input version of JK Latch
- ✅ Toggles output when T = 1
- ✅ Used in counters and clock division

Truth Table

T	Q (Next State)
0	Q (Previous)
1	**Toggles Q**

Applications of Latches:

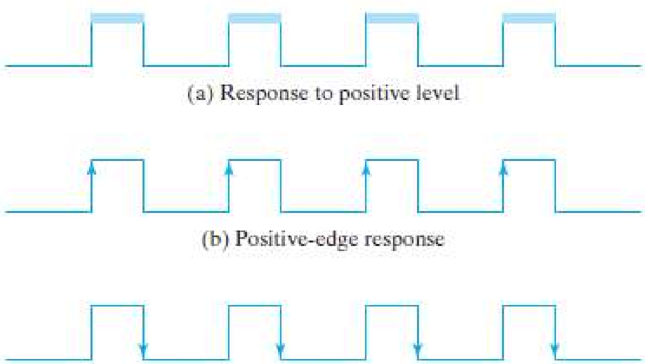
- ✅ Temporary data storage
- ✅ Debounce circuits for switches
- ✅ Memory elements in processors
- ✅ Data synchronization in registers

Sequential Circuits

Flip-Flop:

- The storage elements (memory) used in clocked sequential circuits are called *flip-flops*.
- A flip-flop is a binary storage device capable of storing one bit of information.
- In a stable state, the output of a flip-flop is either 0 or 1.
- Flip-flops are edge-sensitive devices.

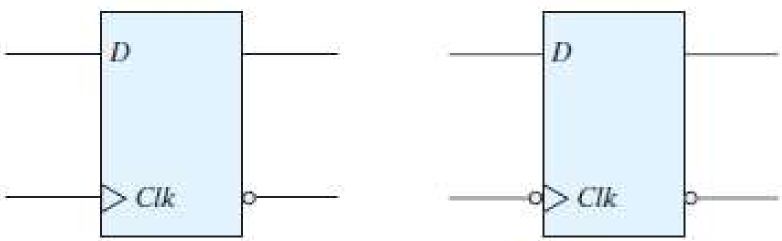
Feature	Latch	Flip-Flop
Triggering	Level-triggered	Edge-triggered
Clock Dependency	No clock required	Requires clock
Speed	Faster	Slower
Usage	Basic memory storage	Registers, counters



(a) Response to positive level

(b) Positive-edge response

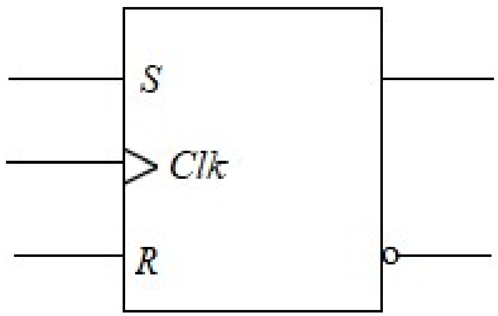
(c) Negative-edge response



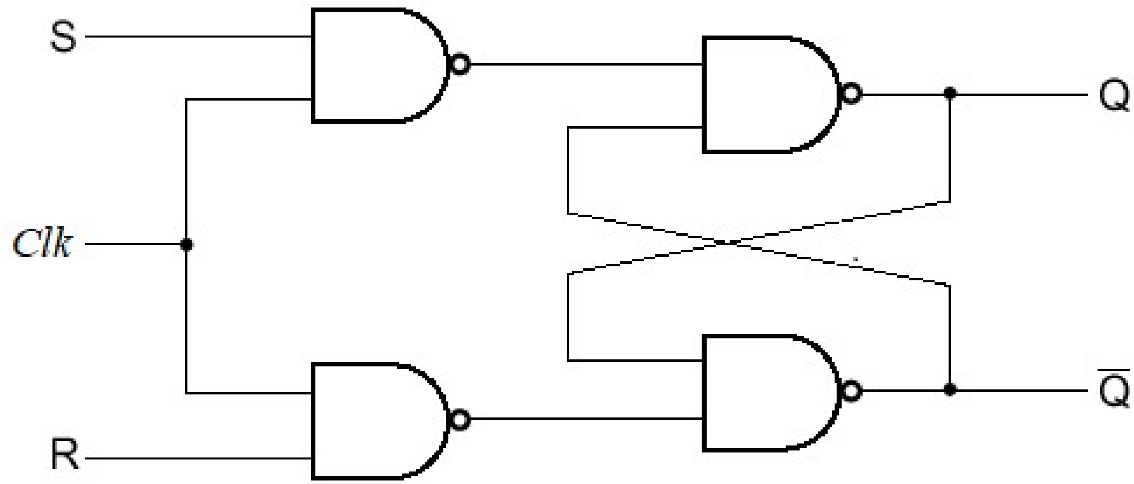
(a) Positive-edge (a) Negative-edge

Sequential Circuits

SR Flip-Flop:



(a) Graphic symbol



(b) Logic diagram

Clk	*S*	*R*	Next state of *Q*	$\overline{Q}$	
0	X	X	Q	$\overline{Q}$	*No change*
1	0	0	Q	$\overline{Q}$	*No change*
1	0	1	0	1	*Reset*
1	1	0	1	0	*Set*
1	1	1	1	1	*Indeterminate*

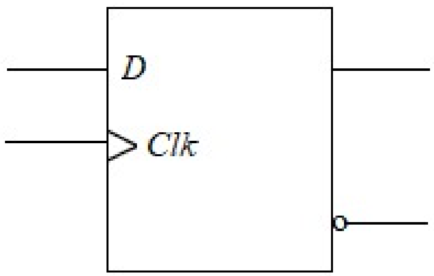
(c) Function table

S	*R*	Q_{n+1}	
0	0	Q_n	*No change*
0	1	0	*Reset*
1	0	1	*Set*
1	1	X	*Indeterminate*

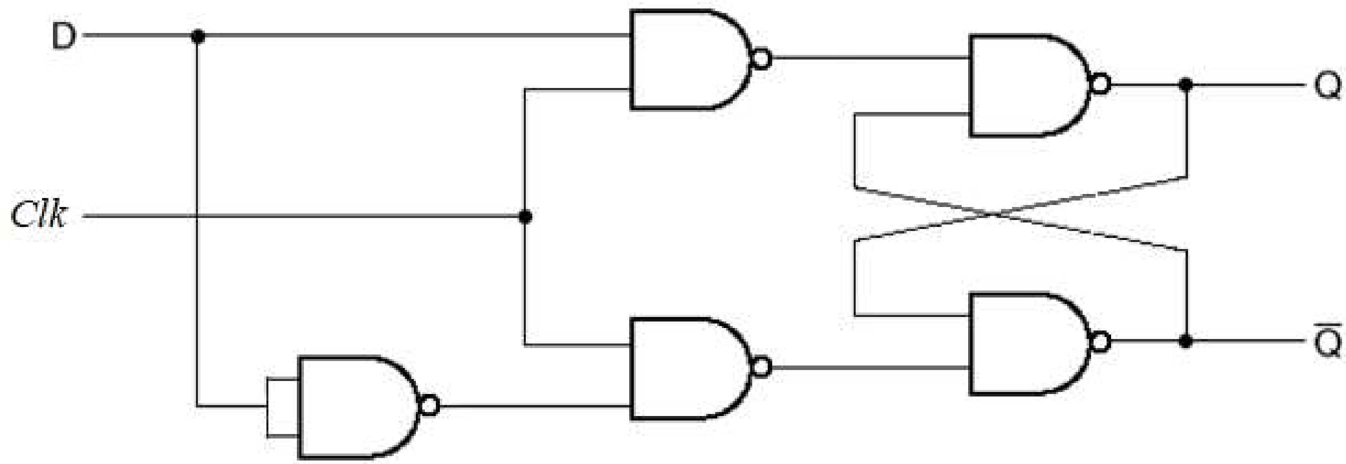
(d) Characteristic table

Sequential Circuits

D Flip-Flop:



(a) Graphic symbol



(b) Logic diagram

Clk	D	Next state of Q	$\overline{Q}$	
0	X	Q	$\overline{Q}$	*No change*
1	0	0	1	*Reset*
1	1	1	0	*Set*

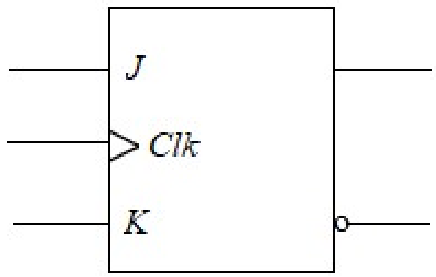
(c) Function table

D	Q_{n+1}	
0	0	*Reset*
1	1	*Set*

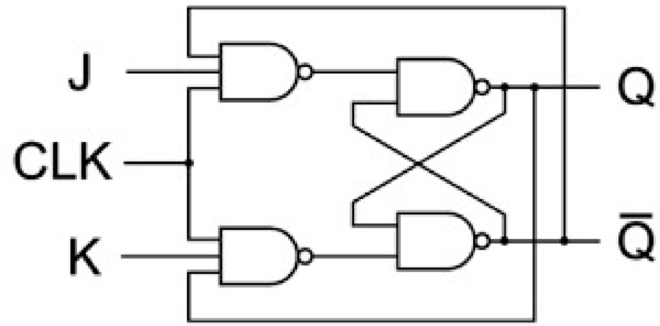
(d) Characteristic table

Sequential Circuits

JK Flip-Flop:



(a) Graphic symbol



(b) Logic diagram

Clk	J	K	Next state of Q	$\overline{Q}$	
0	X	X	Q	Q′	*No change*
1	0	0	Q	Q′	*No change*
1	0	1	0	1	*Reset*
1	1	0	1	0	*Set*
1	1	1	Q′	Q	*Complement*

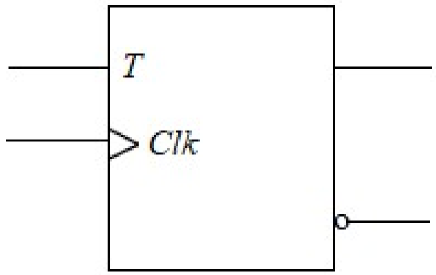
(c) Function table

J	K	Q_{n+1}	
0	0	Q_n	*No change*
0	1	0	*Reset*
1	0	1	*Set*
1	1	$\overline{Q}_n$	*Complement*

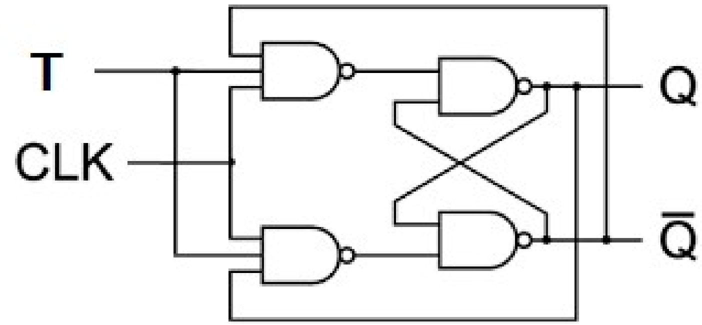
(d) Characteristic table

Sequential Circuits

T Flip-Flop:



(a) Graphic symbol



(b) Logic diagram

Clk	T	Next state of Q	$\overline{Q}$	
0	X	Q	Q'	*No change*
1	0	Q	Q'	*No change*
1	1	Q'	Q	*Complement*

(c) Function table

T	Q_{n+1}	
0	Q_n	*No change*
1	$\overline{Q}_n$	*Complement*

(d) Characteristic table

Applications of Flip-Flops:

- ✅ Registers in processors
- ✅ Data synchronization
- ✅ Frequency division and counters
- ✅ State machines and control logic

Flip-Flop Analysis

- ✓ Characteristic Table: A characteristic table defines the logical properties of a flip-flop by describing its operation in tabular form.

- ✓ Characteristic Equation: The logical properties of a flip-flop, as described in the characteristic table, can be expressed algebraically with a characteristic equation.

- ✓ State Table: The time sequence of inputs, outputs, and flip-flop states can be enumerated in a *state table* (sometimes called a *transition table*).

- ✓ The table consists of four sections labeled *present state*, *input*, *next state*, and *output*.

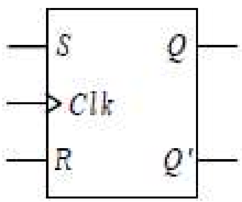
- ✓ State Diagram: The information available in a state table can be represented graphically in the form of a state diagram.

- ✓ Excitation Table: A table that lists the required inputs for a given change of state is called an *excitation table*.

Flip-Flop Analysis

SR Flip-Flop:

Characteristic Table

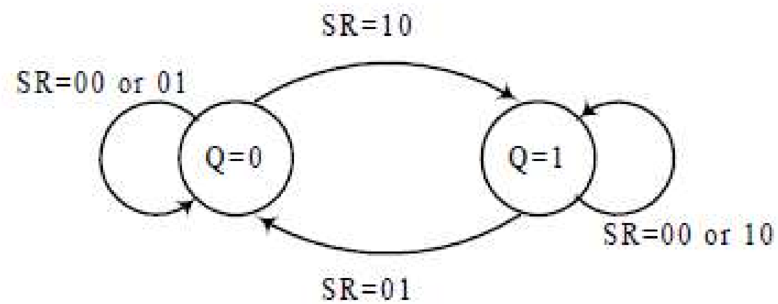


S	R	Q_{n+1}
0	0	Q_n
0	1	0
1	0	1
1	1	X

State Table

S	R	Q_n	Q_{n+1}
0	0	0	0
0	0	1	1
0	1	0	0
0	1	1	0
1	0	0	1
1	0	1	1
1	1	0	×
1	1	1	×

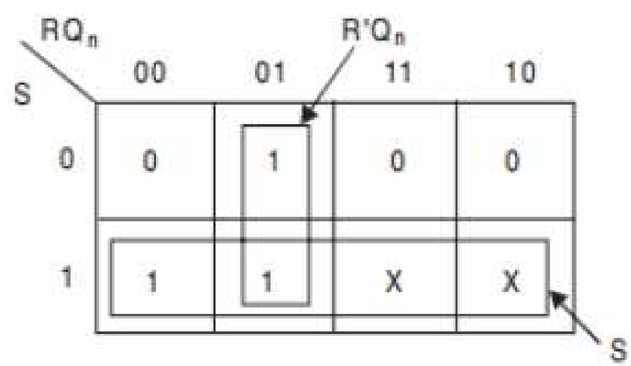
State Diagram / Characteristic Equation



$$Q_{n+1} = S + R'Q_n$$

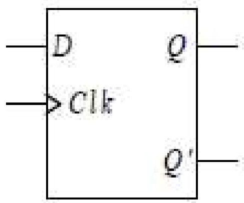
Excitation Table

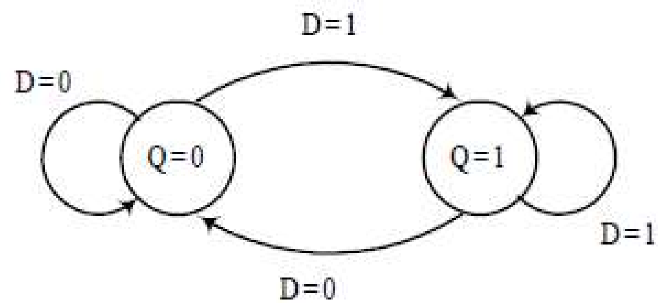
Q_n	Q_{n+1}	S	R
0	0	0	×
0	1	1	0
1	0	0	1
1	1	×	0



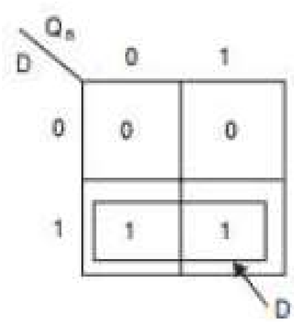
Flip-Flop Analysis

D Flip-Flop:

<table>
<tr><th>Characteristic Table</th><th>State Table</th><th>State Diagram / Characteristic Equation</th><th>Excitation Table</th></tr>
<tr>
<td>


<table>
<tr><th>D</th><th>Q_{n+1}</th></tr>
<tr><td>0</td><td>0</td></tr>
<tr><td>1</td><td>1</td></tr>
</table>
</td>
<td>
<table>
<tr><th>D</th><th>Q_n</th><th>Q_{n+1}</th></tr>
<tr><td>0</td><td>0</td><td>0</td></tr>
<tr><td>0</td><td>1</td><td>0</td></tr>
<tr><td>1</td><td>0</td><td>1</td></tr>
<tr><td>1</td><td>1</td><td>1</td></tr>
</table>
</td>
<td>


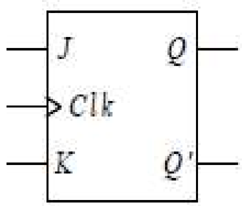
$Q_{n+1} = D$
</td>
<td>
<table>
<tr><th>Q_n</th><th>Q_{n+1}</th><th>D</th></tr>
<tr><td>0</td><td>0</td><td>0</td></tr>
<tr><td>0</td><td>1</td><td>1</td></tr>
<tr><td>1</td><td>0</td><td>0</td></tr>
<tr><td>1</td><td>1</td><td>1</td></tr>
</table>
</td>
</tr>
</table>



Flip-Flop Analysis

JK Flip-Flop:

Characteristic Table

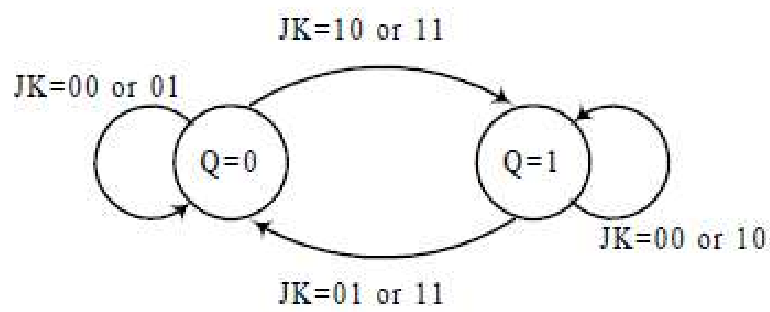


J	K	Q_{n+1}
0	0	Q_n
0	1	0
1	0	1
1	1	$\overline{Q}_n$

State Table

J	K	Q_n	Q_{n+1}
0	0	0	0
0	0	1	1
0	1	0	0
0	1	1	0
1	0	0	1
1	0	1	1
1	1	0	1
1	1	1	0

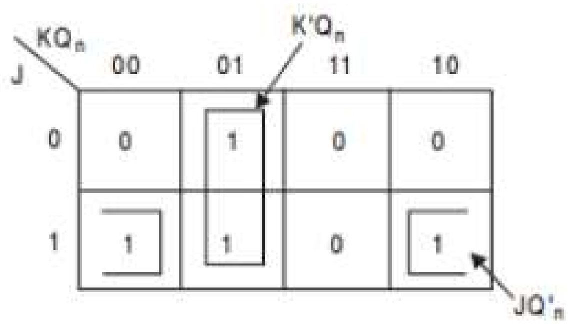
State Diagram / Characteristic Equation



$$Q_{n+1} = JQ'_n + K'Q_n$$

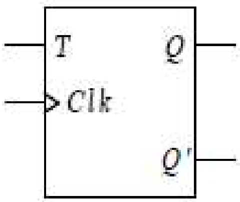
Excitation Table

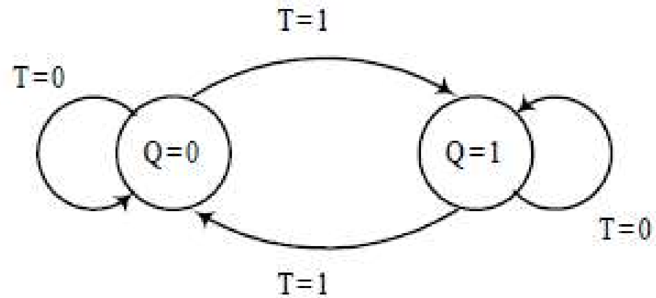
Q_n	Q_{n+1}	J	K
0	0	0	×
0	1	1	×
1	0	×	1
1	1	×	0



Flip-Flop Analysis

T Flip-Flop:

<table>
<tr><th>Characteristic Table</th><th>State Table</th><th>State Diagram / Characteristic Equation</th><th>Excitation Table</th></tr>
<tr>
<td>


<table>
<tr><th>T</th><th>Q_{n+1}</th></tr>
<tr><td>0</td><td>Q_n</td></tr>
<tr><td>1</td><td>$\overline{Q}_n$</td></tr>
</table>
</td>
<td>
<table>
<tr><th>T</th><th>Q_n</th><th>Q_{n+1}</th></tr>
<tr><td>0</td><td>0</td><td>0</td></tr>
<tr><td>0</td><td>1</td><td>1</td></tr>
<tr><td>1</td><td>0</td><td>1</td></tr>
<tr><td>1</td><td>1</td><td>0</td></tr>
</table>
</td>
<td>


$Q_{n+1} = TQ'_n + T'Q_n$
</td>
<td>
<table>
<tr><th>Q_n</th><th>Q_{n+1}</th><th>T</th></tr>
<tr><td>0</td><td>0</td><td>0</td></tr>
<tr><td>0</td><td>1</td><td>1</td></tr>
<tr><td>1</td><td>0</td><td>1</td></tr>
<tr><td>1</td><td>1</td><td>0</td></tr>
</table>
</td>
</tr>
</table>

$T \backslash Q_n$	0	1
0	0	1
1	1	0

Flip-Flop Analysis

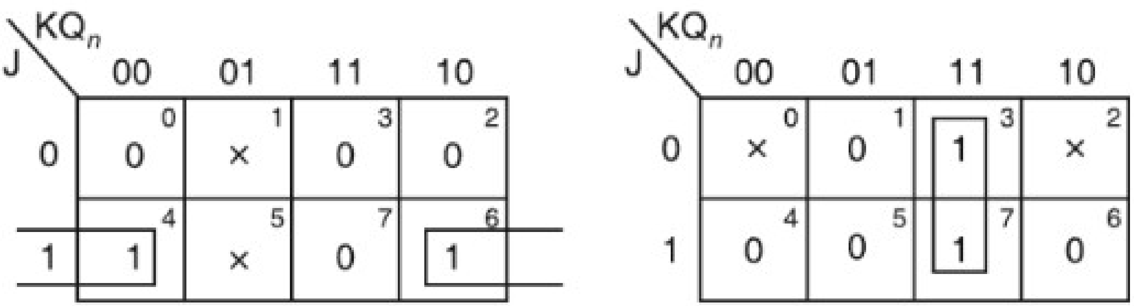
Conversion of Flip-Flops:

- ✓ To convert one type of flip-flop into another type, a combinational circuit is designed such that if the inputs of the required flip-flop are fed as inputs to the combinational circuit and the output of the combinational circuit is connected to the inputs of the actual flip-flop, then the output of the actual flip-flop is the output of the required flip-flop.

Conversion of S-R Flip-flop to J-K Flip-lop.

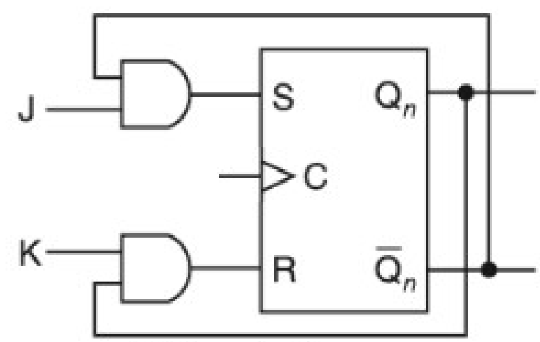
External Inputs		Present State	Next State	Flip-flop Inputs	
J	K	Q_n	Q_{n+1}	S	R
0	0	0	0	0	×
0	0	1	1	×	0
0	1	0	0	0	×
0	1	1	0	0	1
1	0	0	1	1	0
1	0	1	1	×	0
1	1	0	1	1	0
1	1	1	0	0	1

(a) Conversion table



$S = J\overline{Q}_n$ $R = KQ_n$

(b) K-maps for S and R

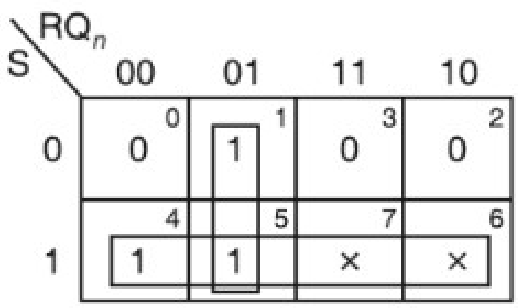


(c) Logic diagram

Conversion of D Flip-flop to S-R Flip-flop

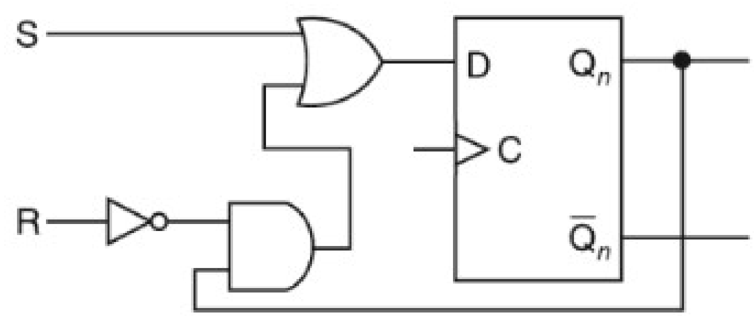
External Inputs		Present State	Next State	Flip-flop Input
S	R	Q_n	Q_{n+1}	D
0	0	0	0	0
0	0	1	1	1
0	1	0	0	0
0	1	1	0	0
1	0	0	1	1
1	0	1	1	1
1	1	0	×	×
1	1	1	×	×

(a) Conversion table



$D = S + \bar{R}Q_n$

(b) K-map for D

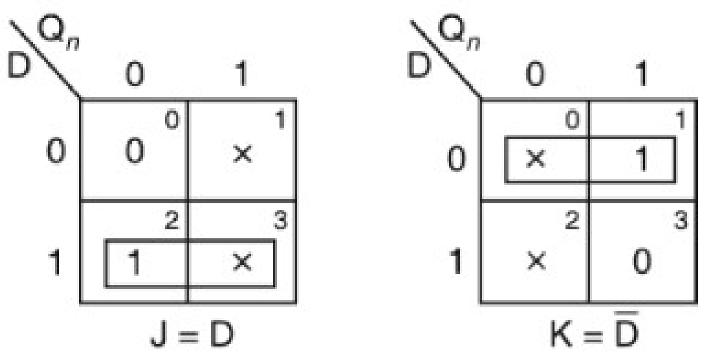


(c) Logic diagram

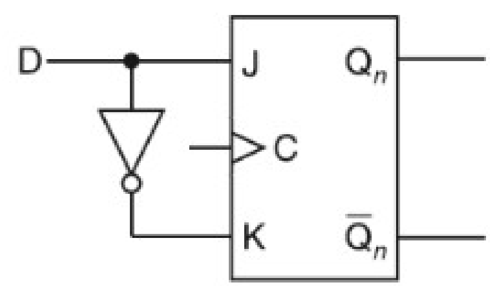
Conversion of J-K Flip-flop to D Flip-flop.

External Input	Present State	Next State	Flip-flop Inputs	
D	Q_n	Q_{n+1}	J	K
0	0	0	0	×
0	1	0	×	1
1	0	1	1	×
1	1	1	×	0

(a) Conversion table



(b) K-maps for J and K

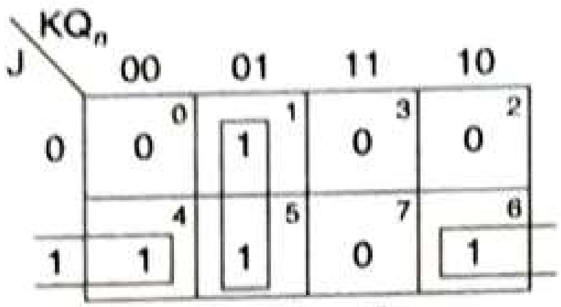


(c) Logic diagram

Conversion of D Flip-flop to J-K Flip-flop

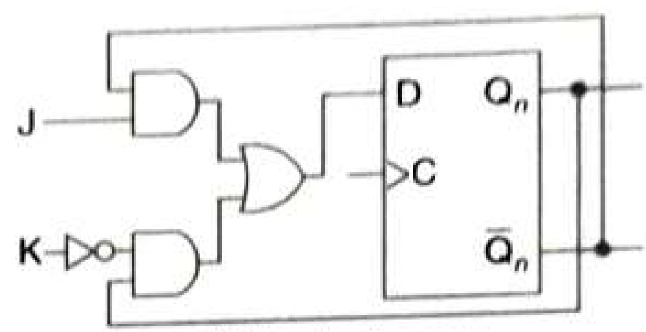
External Inputs		Present State	Next State	Flip-flop Input
J	K	Q_n	Q_{n+1}	D
0	0	0	0	0
0	0	1	1	1
0	1	0	0	0
0	1	1	0	0
1	0	0	1	1
1	0	1	1	1
1	1	0	1	1
1	1	1	0	0

(a) Conversion table



$D = J\bar{Q}_n + \bar{K}Q_n$

(b) K-maps for D



(c) Logic diagram

THANK YOU